



Непрерывная разработка и интеграция CI/CD

...

Лекция 3

Волков Михаил Юрьевич

Тема 3

Обзор Gitlab: установка и настройка

Обзор GitLab

GitLab - веб-приложение и система управления репозиториями программного кода для распределенной системы контроля версий Git.



GitLab

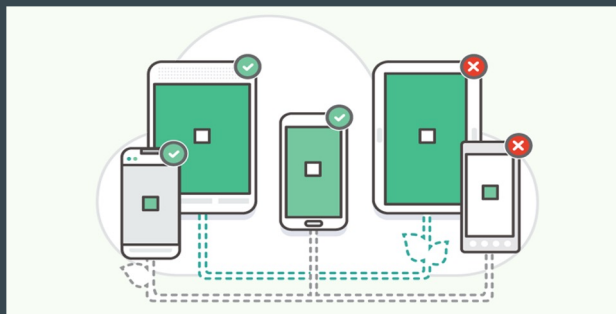
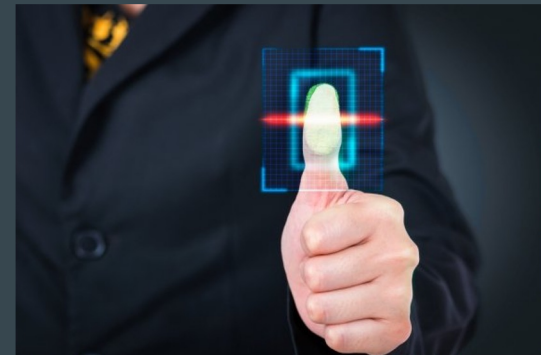
Обзор GitLab

GitLab позволяет взаимодействовать с репозиториями, управлять правами доступа и пользователями, отслеживать ошибки, автоматизировать процессы и выполнять многие другие операции. Установить и использовать его можно на собственном сервере или же в облаке.

Обзор GitLab

Для чего нужен GitLab:

1. Планирование
2. Создание
3. Тестирование
4. Сборка
5. Релиз
6. Конфигурирование
7. Мониторинг



Сравнение Git, GitLab и GitHub

Git представляет собой распределенную систему контроля версий. Она позволяет разработчикам контролировать изменения в файлах и работать совместно с другими специалистами. Git также локально сохраняет весь репозиторий в файл небольшого объема, не снижая качества данных.

GitHub, как и GitLab, представляет собой онлайн-сервис для размещения репозиториев, удаленного управления ими и других задач разработки. В нем предусмотрены багтрекинг, вики для каждого проекта, история коммитов, графика, вложенные списки задач и многое другое.



git



GitHub

Сравнение Git, GitLab и GitHub



GitLab



GitHub

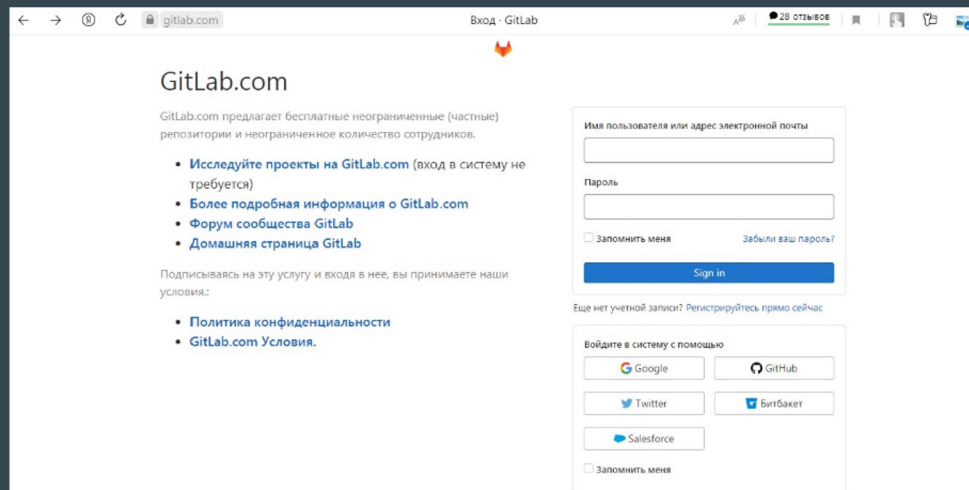
Основные отличия GitLab и GitHub:

1. В GitLab реализована встроенная бесплатная непрерывная интеграция. В GitHub есть инструмент Actions, который позволяет запускать бесплатные непрерывные интеграции в публичных репозиториях.
2. GitLab задействует Kubernetes для бесппроблемного развертывания. В GitHub встроенной платформы развертывания нет.
3. В GitLab есть бесплатные репозитории частного формата для проектов, имеющих открытый исходный код. GitHub такого не имеет.

Основные этапы при работе с GitLab

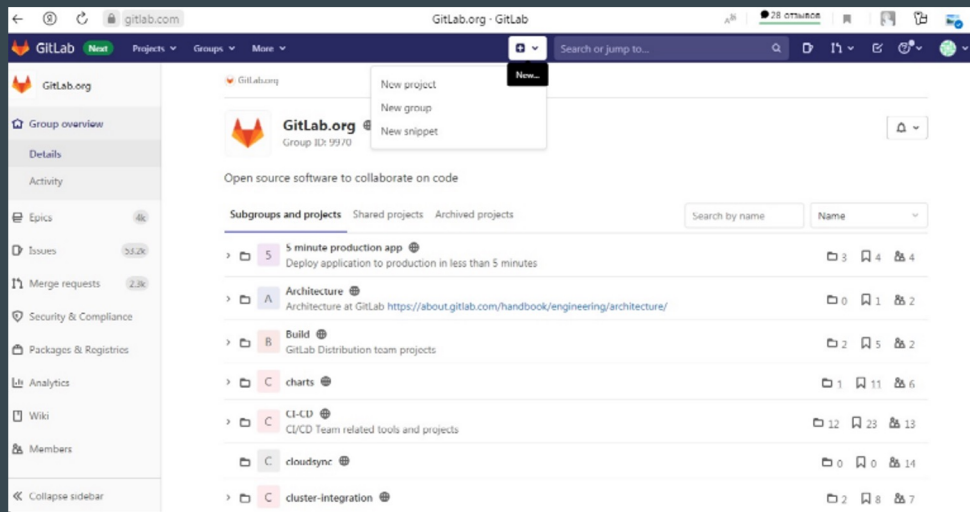
1) Создание аккаунта

GitLab имеет простую процедуру регистрации. На главной странице официального сайта есть форма входа, в которой надо ввести только имя пользователя или адрес электронной почты и придумать пароль.



The screenshot shows the GitLab.com login page. At the top, the browser address bar displays 'gitlab.com' and the page title is 'Вход · GitLab'. The main heading is 'GitLab.com', followed by a subtext: 'GitLab.com предлагает бесплатные неограниченные (частные) репозитории и неограниченное количество сотрудников.' Below this, there are three links: 'Исследуйте проекты на GitLab.com (вход в систему не требуется)', 'Более подробная информация о GitLab.com', 'Форум сообщества GitLab', and 'Домашняя страница GitLab'. A note states: 'Подписываясь на эту услугу и входя в нее, вы принимаете наши условия:'. Below this are two links: 'Политика конфиденциальности' and 'GitLab.com Условия.'. On the right side, there is a login form with fields for 'Имя пользователя или адрес электронной почты' and 'Пароль'. There are checkboxes for 'Запомнить меня' and a link 'Забыли ваш пароль?'. A blue 'Sign in' button is at the bottom of the form. Below the login form, there is a link 'Еще нет учетной записи? Регистрируйтесь прямо сейчас'. At the bottom, there is a section 'Войдите в систему с помощью' with buttons for Google, GitHub, Twitter, Bitbucket, and Salesforce. There is also a 'Запомнить меня' checkbox at the bottom of this section.

Основные этапы при работе с GitLab



2) Создание репозитория

Для создания нового проекта надо нажать на значок «+» по центру экрана и выбрать соответствующий пункт.

Основные этапы при работе с GitLab

3) Загрузка файлов проекта

В интерфейсе предусмотрены удобные варианты загрузки проектов. На главной странице репозитория можно загрузить файл, создать новый файл, добавить лицензию и файл Readme. При этом загрузка файлов с компьютера выполняется быстро, не требует переформатирования или других операций.

4) SSH-ключи - Чтобы во время загрузки данных репозитория не приходилось вводить логин и пароль, для авторизации можно использовать SSH-ключи.

Основные этапы при работе с GitLab

5) Ветки репозитория

По умолчанию в репозитории GitLab предусмотрена только одна ветка — `master(main)`.

В веб-интерфейсе сервиса ветки отображаются слева, что упрощает переход между ними. Ветки создаются в пару кликов — нужно выбрать «+» по центру экрана и нажать «New branch». Кроме того, после обновления изменений в репозитории в GitLab отображаются и новые ветки, созданные в Git.

Основные этапы при работе с GitLab

6) Слияние веток

В ветках разрабатывается функциональность, поэтому может потребоваться их перенос — для этого предназначены запросы слияния («Merge request gitlab»). Для использования этой возможности в интерфейсе GitLab нужно нажать кнопку «Create merge request», задать описание «Merge Request», выбрать исходную и целевые ветки. После одобрения запроса на слияние надо нажать на кнопку «Merge». В результате файлы ветки преемника будут заменены файлами из ветки источника.

Основные этапы при работе с GitLab

7) Добавление пользователей

В GitLab можно добавлять неограниченное количество разработчиков даже к приватным репозиториям. Чтобы сделать это, надо перейти в меню «Настройки» (Settings) и выбрать пункт «Участники» (Members). В этом пункте в поле «Выбрать участника для приглашения» (Select members to invite) надо указать адрес электронной почты пользователя или его никнейм. Перед отправкой приглашения также указывается уровень доступа. Для добавления надо нажать «Добавить в проект» (Add to project).

8) Удаление проекта

Для удаления проекта нужно перейти в «Настройки» (Settings) -> «Главные» (General) -> «Продвинутые» (Advanced) и выбрать пункт «Удалить проект» (Remove Project). Перед удалением проекта потребуется ввести его имя.

Возможные проблемы при работе с GitLab

При работе с GitLab могут возникать проблемы, для каждой из которых есть решение:

- 1) Ошибка при выполнении PULL-запросов: Причиной может быть длительная фаза SSH-аутентификации или низкая скорость обработки запросов. Решается перенастройкой репозитория.
- 2) Ошибка при входе: Может появляться после некорректного введения логина и пароля учетной записи. Решается удалением данных через панель управления.
- 3) Ошибка при запуске: Система может указывать на отсутствие GitLab-серверов по заявленным IP-адресам. Причиной проблемы может быть неверное введение команды запуска, использование динамического IP или ошибки при назначении порта. Решается перенастройкой.

Заключение по разделу

Веб-приложение GitLab является отличным решением для построения рабочих процессов CI/CD в облаке, в том числе если системы контроля и разработки надо установить на личном сервере.

GitLab имеет множество сфер применения и широкие возможности, что в сочетании с удобным инструментарием делает его удобным сервисом как для начинающих разработчиков, так и для профессионалов.

Установка и настройка GitLab на Linux Fedora

Для установки и настройки GitLab в ОС Fedora необходимо выполнить несколько шагов:

1. Проверьте версию установленной ОС Fedora;
2. Установите расширение «curl polycoreutils openssh-server perl»;
3. Включите и установите sshd status;
4. Проверьте статус sshd;
5. Запустите firewall service для http и https;
6. Перезагрузите firewall service;
7. Проверьте статус firewall;

Установка и настройка GitLab на Linux Fedora

8. Установите postfix;
9. Включите и запустите сервис postfix;
10. Установите расширение gitlab-ce;
11. Установите расширение gitlab-ce rpm;
12. Настройте gitlab-ce;
13. Предоставьте доступ к GitLab через браузер.

На этом установка и настройка GitLab в OS Fedora завершена.

CI/CD в GitLab

Перед началом интеграции CI/CD в GitLab, необходимо убедиться:

1. Есть необходимый проект в GitLab, в который необходимо интегрировать CI/CD;
2. Ваша роль в проекте сопровождающего или же владельца.

CI/CD в GitLab

Для настройки и запуска первого pipeline необходимо:

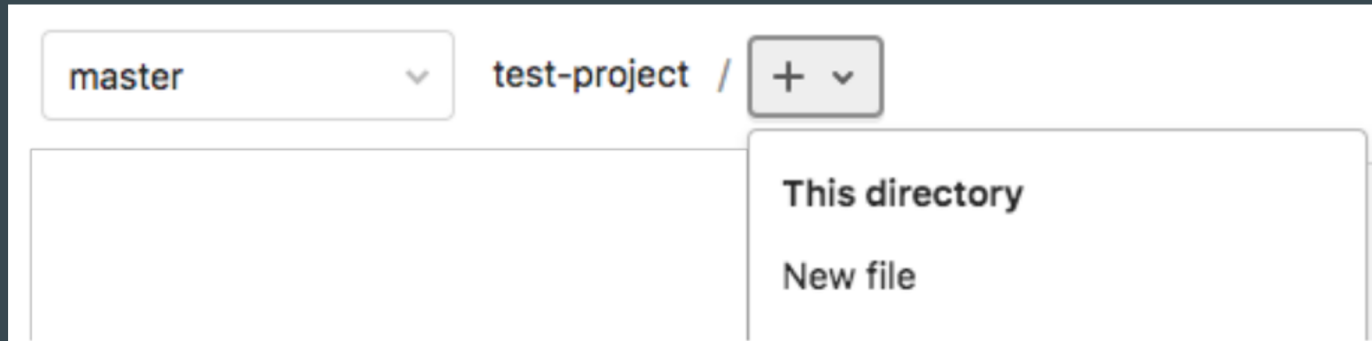
1. Создайте файл `.gitlab-ci.yml` в корне вашего репозитория. В этом файле вы определяете задания CI/CD.

Когда вы фиксируете файл в своем репозитории, программа runner запускает ваши задания. Результаты задания отображаются в конвейере.



CI/CD в GitLab

2. Над списком файлов выберите ветку, которую вы хотите зафиксировать. Если вы не уверены, оставьте master или main. Затем выберите значок плюса (+) и New file:



CI/CD в GitLab

3. В качестве имени файла должно быть .gitlab-ci.yml и в окне вставьте код.

```
build-job:
  stage: build
  script:
    - echo "Hello, $GITLAB_USER_LOGIN!"
test-job1:
  stage: test
  script:
    - echo "This job tests something"
test-job2:
  stage: test
  script:
    - echo "This job tests something, but takes more time
than test-job1."
    - echo "After the echo commands complete, it runs the
sleep command for 20 seconds"
    - echo "which simulates a test that runs 20 seconds longer
than test-job1"
    - sleep 20
deploy-prod:
  stage: deploy
  script:
    - echo "This job deploys something from the
$CI_COMMIT_BRANCH branch."
  environment: production
```

CI/CD в GitLab

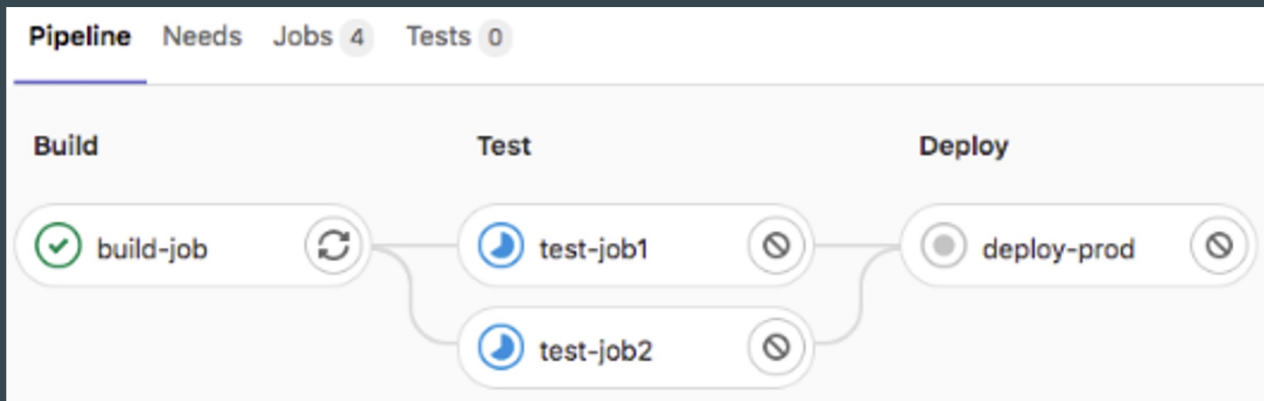
4. Выбрать commit changes - Конвейер запускается и выполняет задания, определенные в файле .gitlab-ci.yml.

Для просмотра конвейера и его содержимого, нужно перейти в CI/CD > Pipelines.

Status	Pipeline	Triggerer	Commit	Stages
 running	#217408060 latest		 master -> 271ad0cd  Update .gitlab-ci.yml	

CI/CD в GitLab

Можно посмотреть визуальное представление конвейера, выбрав идентификатор конвейера.



CI/CD в GitLab

```
✓ passed Job #855275091 triggered 23 minutes ago by Suzanne Selhorn
```

```
1 Running with gitlab-runner 13.6.0-rc1 (d83ac56c)
2   on docker-auto-scale ed2dce3a
3   ✓ Preparing the "docker+machine" executor
4   Using Docker executor with image ruby:2.5 ...
5   Pulling docker image ruby:2.5 ...
6   Using docker image sha256:b7280b81558d31d64ac82aa66a9540e04baf9d15abb8fff
   ed62cd60e4fb5bf4132943d6fa2688 ...
7   ✓ Preparing environment
8   Running on runner-ed2dce3a-project-16381496-concurrent-0 via runner-ed2dc
9   ✓ Getting source from Git repository
10  $ eval "$CI_PRE_CLONE_SCRIPT"
11  Fetching changes with git depth set to 50...
12  Initialized empty Git repository in /builds/sselhorn/test-project/.git/
13  Created fresh repository.
14  Checking out 7353da73 as master...
15  Skipping Git submodules setup
16  ✓ Executing "step_script" stage of the job script
17  $ echo "This job deploys something from the $CI_COMMIT_BRANCH branch."
18  This job deploys something from the master branch.
19  ✓ Cleaning up file based variables
20  Job succeeded
```

Для просмотра подробных сведений о задании, нужно выбрать название задания. Например, deploy-prod.

Спасибо за внимание !

